

Accelerating Streaming Video Large Language Models via Hierarchical Token Compression

Yiyu Wang^{1*} Xuyang Liu^{1,2*,†} Xiyan Gui^{1,3} Xinying Lin⁴ Boxue Yang¹

Chenfei Liao^{1,5} Tailai Chen¹ Linfeng Zhang^{1✉}

¹EPIC Lab, Shanghai Jiao Tong University ²Sichuan University

³Huazhong University of Science and Technology ⁴Sun Yat-sen University

⁵Hong Kong University of Science and Technology (Guangzhou)

Code: <https://github.com/lern-to-write/STC>

Abstract

Streaming Video Large Language Models (VideoLLMs) have demonstrated impressive performance across various video understanding tasks, but they face significant challenges in real-time deployment due to the high computational cost of processing dense visual tokens from continuous video streams. In streaming video scenarios, the primary bottleneck lies in the Vision Transformer (ViT) encoding stage, where redundant processing of temporally similar frames leads to inefficiency. Additionally, inflated token sequences during LLM pre-filling further exacerbate latency and memory overhead. To address these challenges, we propose **Streaming Token Compression (STC)**, a plug-and-play hierarchical framework that seamlessly integrates into existing streaming VideoLLMs, optimizing both ViT encoding and LLM pre-filling stages to accelerate processing. STC introduces two token-level accelerators: **STC-Cacher**, which reduces ViT encoding overhead by caching and reusing features from temporally similar frames, and **STC-Pruner**, which compresses the visual token sequence before it enters the LLM, preserving only the most salient tokens based on both spatial and temporal relevance. Extensive experiments on four baseline streaming VideoLLMs across five benchmarks demonstrate that STC outperforms other compression methods. Notably, STC retains up to **99%** of accuracy on the ReKV framework while reducing ViT encoding latency and LLM pre-filling latency by **24.5%** and **45.3%**.

1. Introduction

Video large language models (VideoLLMs) [8, 45, 46, 50, 57, 58, 62] have demonstrated strong performance across diverse video understanding tasks [16, 21, 51, 64]. Recently,

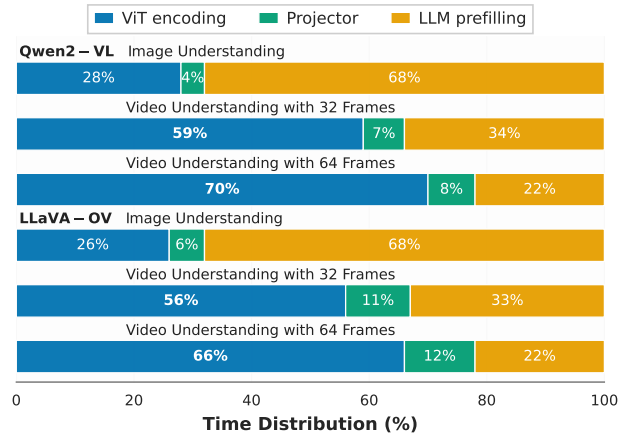


Figure 1. Inference time breakdown across components in various vision-language understanding scenarios. ViT encoding typically accounts for a substantial fraction of the inference time in video understanding, about 2-3 times that in image understanding.

emerging applications such as live sports commentary [38] and augmented reality glasses [49] have created an urgent demand for streaming video understanding (SVU) [17], where models must process incoming video frames continuously and generate responses with minimal latency [4, 36]. However, the computational cost of processing dense visual tokens from multi-frame inputs renders existing VideoLLMs too slow for real-time streaming scenarios, severely limiting their deployment in latency-sensitive applications [28].

To address this, two main approaches have been explored. Token compression methods reduce visual tokens either before [9, 13, 27, 55] or within the LLM [5, 47, 48], improving both prefilling and decoding efficiency. Key-Value (KV) cache compression methods [15, 25] evict less important KV pairs to reduce memory overhead during decoding. However, in streaming scenarios where frames arrive continuously, the computational bottleneck lies primarily in the vision encoder (ViT [14]). Since each incoming frame must be encoded

*Equal contribution. † Project leader. ✉ Corresponding author

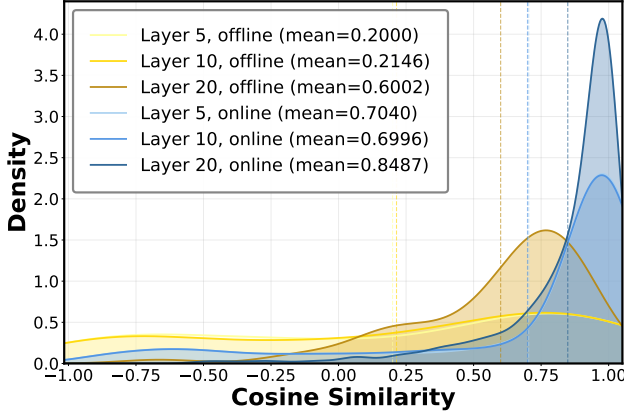


Figure 2. **Temporal redundancy in adjacent frames in ViT encoding.** Streaming videos (“online”) tend to show higher similarity than offline videos, indicating higher temporal redundancy.

independently through ViT, the repeated ViT inference dominates overall latency before tokens even reach the LLM.

Figure 1 demonstrates this phenomenon: the inference time breakdown across components reveals a striking difference between image and video understanding. For models such as Qwen2-VL [44] and LLaVA-OV [19], ViT encoding in video understanding incurs **2-3 times** higher computational cost compared to image understanding, making it the dominant latency contributor. Moreover, when LLaVA-OV processes a 32-frame video, this results in $32 \times 196 = 6,272$ visual tokens fed into the LLM during prefilling. In comparison, image understanding tasks such as MMBench [30] typically require the LLM to process only around 1,900 visual tokens. The over **three-fold** increase translates directly to substantially longer latency, presenting a critical barrier for real-time streaming deployment. These observations underscore that *compression methods must reduce both ViT encoding overhead and context sequence length to enable efficient streaming video understanding*. To better understand the unique characteristics of streaming scenarios and guide our method design, we conduct an empirical analysis comparing streaming and offline video understanding, revealing **two key features** in streaming videos:

(I) Temporal Redundancy in ViT Encoding: Streaming videos require denser frame sampling, leading to substantial temporal redundancy. Figure 2 compares streaming (0.5fps) and offline videos (64 frames) during ViT encoding by LLaVA-OV [19]. At deeper layers (e.g., Layer 20), the mean cosine similarity of adjacent frame features in streaming reaches 0.85, compared to 0.60 in offline settings. This difference arises because *consecutive frames in streaming often capture nearly identical visual content*. However, existing methods fail to exploit this redundancy, as most ignore it during ViT encoding and operate only on final token representations. A few methods, such as ToMe [3], perform token merging within ViT layers, but this disrupts encoding, leading to performance degradation for SVU (see Table 1, 2).

(II) Incomplete Video and Unknown Instructions: Streaming scenarios impose a constraint: *models cannot access complete video content in advance, nor can they know user instructions beforehand*. This makes existing compression strategies ineffective. Methods relying on global video features for token selection [27, 31, 39] cannot operate efficiently, as they require the entire video for compression decisions. Instruction-aware approaches [7, 52, 63], which prune tokens based on query relevance, fail in streaming, where queries arrive only after frames are processed [56]. These challenges highlight a gap: streaming video understanding demands compression methods that operate **causally**, exploiting temporal redundancy during encoding while adapting to incrementally arriving frames, without relying on global context or future instructions.

Based on the above analysis, we propose Streaming Token Compression (STC), a plug-and-play hierarchical acceleration framework that jointly optimizes the ViT encoding and LLM prefilling stages for efficient SVU. STC consists of two *orthogonal* modules. First, **STC-Cacher** reduces ViT forward passes by identifying temporally redundant frames through spatial feature similarity. It caches key frame visual features and reuses them for similar frames, avoiding redundant computations. Second, **STC-Pruner** compresses the visual token sequence after the vision encoder, preserving tokens salient in both spatial and temporal dimensions, and pruning redundant ones to shorten the LLM prefill sequence. Together, these modules enable low-latency video understanding while maintaining high semantic fidelity. Our contributions are **four-fold**:

- **Empirical Analysis of Streaming Scenarios:** We empirically analyze limitations of existing compression methods for SVU, revealing the need for causal compression that reduces ViT cost without relying on future instructions.
- **Plug-and-Play Acceleration Framework:** We propose Streaming Token Compression (STC), a plug-and-play framework that seamlessly integrates into existing VideoLLMs for efficient streaming video understanding.
- **Two Complementary Token Compressors:** STC introduces two modules: STC-Cacher accelerates ViT encoding by caching features from adjacent frames, and STC-Pruner accelerates LLM prefill by pruning low-saliency tokens.
- **Comprehensive Validation and Results:** Extensive experiments demonstrate STC achieves superior performance-efficiency trade-offs, retaining 99% accuracy on ReKV while reducing LLM prefill latency by 45.3%.

2. Related Work

2.1. Streaming Video Understanding

Streaming Video Understanding (SVU) requires VideoLLMs [1, 19, 45, 61, 62] to continuously process incoming video content and respond to user queries in real-

time [6, 12, 24, 34]. SVU systems can generally be categorized into two main paradigms: (i) *End-to-end online models* trained specifically for streaming, such as Dispider [36], which decouples perception and decision-making, and StreamForest [60], which utilizes a Persistent Event Memory Forest for long-term memory retention and a Fine-grained Spatiotemporal Window for real-time comprehension. LiveCC [6] combines streaming ASR transcripts and video frames for continuous training and real-time commentary. While these models are effective in many contexts, they come with high training costs and offer limited compatibility with offline VideoLLMs, making them less efficient for broader use cases. (ii) *Offline-to-online frameworks* adapt pre-trained VideoLLMs [1, 19, 44] to streaming environments without the need for retraining [33, 54, 56]. ReKV [12] pioneers this approach by maintaining frame-wise KV caches and employing sliding-window encoding for real-time video question answering. LiveVLM [33] and StreamChat [24] focus on preserving temporal coherence through effective memory management, enhancing the overall streaming experience.

Despite their success, these approaches still process each frame through ViT encoding, feeding dense token sequences into the LLM, resulting in high computational and memory costs that limit real-time deployment.

2.2. Token Compression for VideoLLMs

Token compression has emerged as a promising strategy to accelerate VideoLLM inference by reducing redundant visual tokens [7, 20, 29, 39, 40, 42, 53, 55]. Existing methods can be broadly categorized into two approaches for VideoLLMs: (i) *Offline compression*, which assumes full access to the entire video. Methods like ToMe [2] merge similar tokens during ViT encoding, while post-encoding methods select salient tokens after feature extraction [7, 18, 55]. The former accelerates both ViT encoding and LLM prefilling, while the latter only accelerates prefilling, leaving ViT encoding inefficient. More critically, methods like SparseVLM [63], DyCoke [42], and VidCom² [26] either rely on global visibility of the entire video or require user input, making them incompatible with causal streaming constraints. (ii) *Streaming compression* adapts compression for streaming settings but remains limited. LiveVLM [33] compresses the KV cache to reduce memory usage but still cannot address the high cost of ViT encoding and long sequence lengths. TimeChat-Online [59] drops tokens based on adjacent-frame similarity, but this short-horizon strategy fails to capture long-range redundancy and often retains repetitive content.

In summary, prior works either overlook the causal nature of streaming or optimize only one stage. Our work is **the first** to jointly optimize ViT encoding and LLM prefill with a streaming-native, two-stage design that respects temporal causality while maximizing redundancy removal.

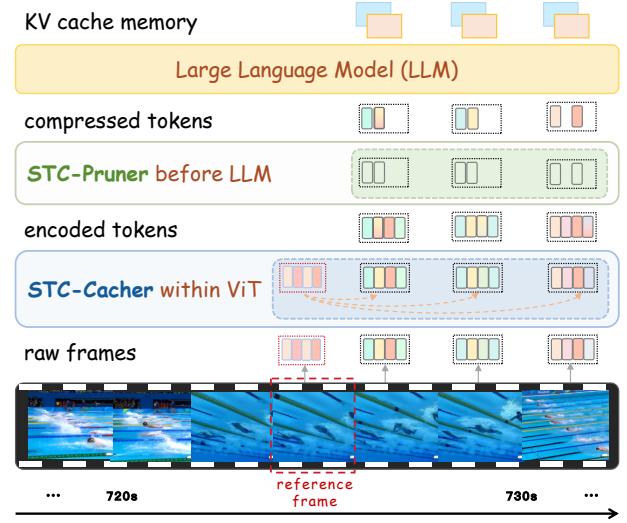


Figure 3. **Overview of Streaming Token Compression (STC).** Our framework accelerates streaming Video-LLMs in two stages. **STC-Cacher** employs selective recomputation to reduce computational redundancy in the ViT. **STC-Pruner** then reduces the token sequence to alleviate the prefilling latency for the LLM.

3. Methodology

3.1. Preliminary

Streaming Inference with VideoLLMs. Most VideoLLMs, initially designed for offline video understanding (OVU) [19, 37, 41, 61, 62], can be adapted for SVU by processing video in discrete chunks. A continuous video stream $\mathbf{V} = \{\mathbf{v}_t\}_{t=1}^T \in \mathbb{R}^{T \times H \times W \times 3}$ is divided into segments, each corresponding to a chunk $\mathbf{V}_c = \{\mathbf{v}_t\}_{t=cL+1}^{(c+1)L}$, where L is the chunk length. For each chunk, a vision encoder (e.g., ViT) extracts visual features as token embeddings $\mathbf{Z}_c = \{\mathbf{z}_t\}_{t=cL+1}^{(c+1)L} \in \mathbb{R}^{L \times N \times D}$, where N is the number of patches and D is the embedding dimension.

These token embeddings $\mathbf{Z}_c$ are projected into the LLM’s embedding space for autoregressive processing. The LLM uses visual tokens $\mathbf{X}_c^v$ and any prior textual context $\mathbf{X}_c^t$ to predict subsequent tokens. To maintain temporal consistency under causal constraints, KV states generated from each chunk are cached in a memory bank $\mathcal{M}$, which serves as context for future processing. Specifically, the KV states $\mathbf{KV}_c = \{\mathbf{K}_c, \mathbf{V}_c\}$, where $\mathbf{K}_c$ and $\mathbf{V}_c$ represent key and value embeddings, are stored and used to condition the model’s future generations.

This approach enables continuous video processing while respecting causal constraints, supporting real-time video understanding in streaming settings.

The Redundancy Bottleneck. This sequential processing paradigm introduces computational bottlenecks. We identify two main sources of redundancy:

- **Temporal Redundancy in ViT Encoding:** In streaming video settings, adjacent frames often share similar visual

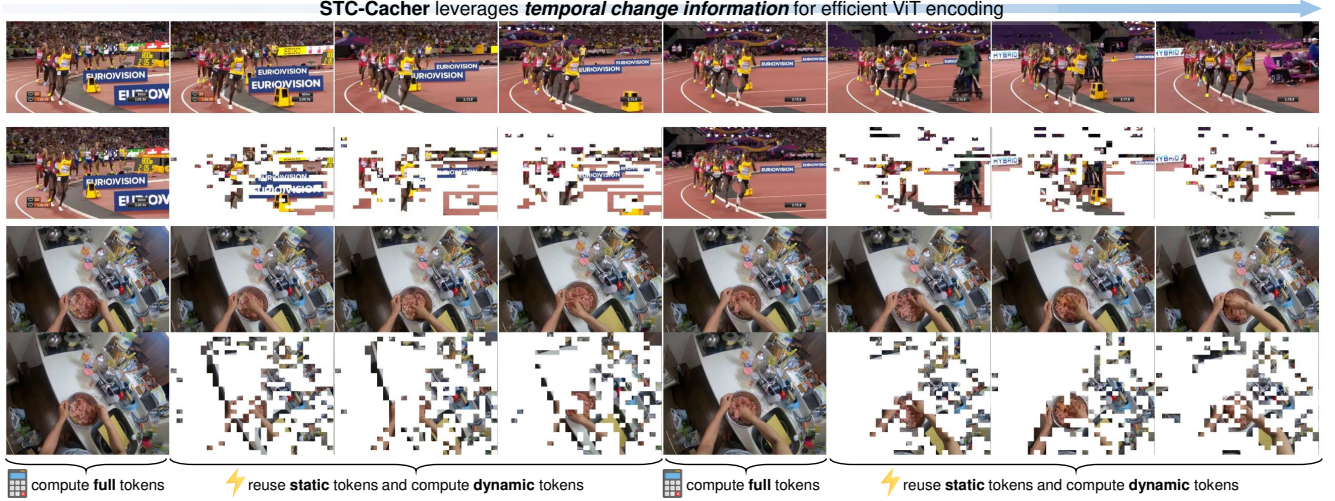


Figure 4. **Visualization of cache-aware selective computation by STC-Cacher.** For reference frames, STC-Cacher computes and caches all tokens. For subsequent frames, only dynamic tokens are computed, while static tokens reuse cached features from reference frames.

content, such as static backgrounds, leading to redundant computations in ViT encoding. Processing static information provides diminishing returns in new insights.

- **Long Context Redundancy in LLM Prefilling:** As the video progresses, redundancy accumulates in the token sequence fed to the LLM. This includes persistent temporal features (e.g., background) and low-information regions within frames. This redundancy burdens the LLM prefilling stage, characterized by the quadratic computational complexity $\mathcal{O}(N^2)$ of self-attention [43], and inflates the memory footprint of the KV cache $\mathcal{M}$ without proportional gains in comprehension.

These observations motivate a unified compression strategy to reduce redundancy at both the vision encoding and language processing stages, while adhering to the causal constraints of streaming inference.

3.2. Streaming Token Compression

To address the redundancy bottlenecks discussed above, we propose **Streaming Token Compression (STC)**, a plug-and-play acceleration framework for efficient SVU. As shown in Figure 3, STC tackles redundancy through two complementary token-level compression components:

- **STC-Cacher** combines *temporal change information* to **cache** and reuse visual features for static content, selectively computing only the temporally dynamic tokens, thus addressing temporal redundancy in ViT encoding.
- **STC-Pruner** combines *causal event information* to **prune** redundant tokens in the visual token sequence before it enters the LLM, ensuring precise token sequence compression to reduce computational cost.

Both components are **query- and future-agnostic**, ensuring compatibility with real-time streaming constraints. Together, they reduce computational overhead while preserving causal integrity for SVU. Below, we elaborate on the

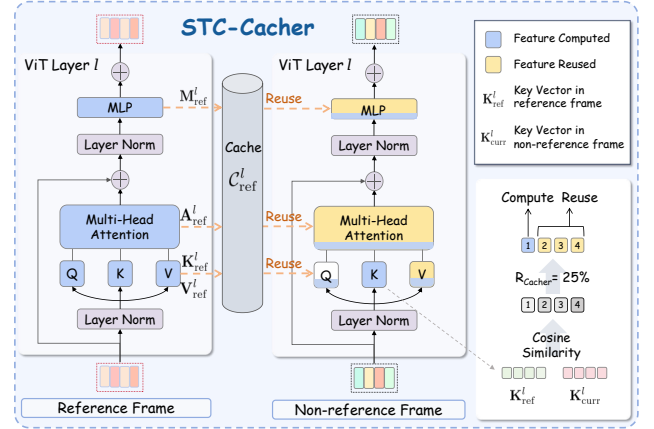


Figure 5. **The Mechanism of STC-Cacher.** Instead of a full forward pass, STC-Cacher identifies novel tokens by comparing their Key projections (K_{curr}) to a cached reference (K_{ref}). It then selectively recomputes only the Query and Value representations for these dynamic tokens and **scatters** Value into the cached Value matrix for an efficient, low-rank update attention mechanism.

detailed operations of these two components.

3.3. STC-Cacher: Selective Computation in ViT

The computational cost of ViT encoding in streaming scenarios is primarily driven by the repetitive processing of temporally redundant frames. While naive strategies like token pruning or merging [3, 22] reduce computational cost, they result in significant information loss during the visual encoding process. This leads to our first research question:

How can we reduce the computational cost of ViT encoding on sequential frames while preserving temporal information content from the original video stream?

To address this, we introduce **STC-Cacher** (Figure 5), a **cache-aware selective computation** strategy designed to

focus computation on dynamic tokens while reusing cached states for static content. The process is detailed below:

(I) Reference Frame (Full Computation and Caching): For a reference frame $\mathbf{f}_{\text{ref}}$, we perform a full forward pass, caching intermediate representations at each ViT layer to serve as a reference for subsequent frames. The cached information $\mathcal{C}_{\text{ref}}^l$ at layer l includes:

$$\mathcal{C}_{\text{ref}}^l = \{\mathbf{K}_{\text{ref}}^l, \mathbf{V}_{\text{ref}}^l, \mathbf{A}_{\text{ref}}^l, \mathbf{M}_{\text{ref}}^l\} \quad (1)$$

These representations are used as reference when processing non-reference frames.

(II) Non-reference Frame (Selective Computation): For subsequent frames $\mathbf{F}_{\text{new}}$, we leverage the cached information $\mathcal{C}_{\text{ref}}^l$ to bypass most computations. Instead of a full forward pass, we focus on the **dynamic tokens**, which contain novel information compared to the cached reference.

We introduce two hyper-parameters: the *cache interval* $\mathcal{N}$ and the *cache reuse ratio* R_{Cacher} . The cache interval $\mathcal{N}$ determines how frequently the reference frame is updated, and R_{Cacher} controls the proportion of dynamic tokens selected. For instance, with $\mathcal{N} = 4$ and $R_{\text{Cacher}} = 75\%$, every 4th frame is chosen as a reference, and 25% of tokens are computed as dynamic, while the remaining 75% are reused from the cache. The process is as follows:

(i) Identify Dynamic Tokens: We compute the cosine similarity between the current Key projections $\mathbf{K}_{\text{curr},\mathbf{f}}^l$ and the cached reference $\mathbf{K}_{\text{ref}}^l$:

$$S_{\mathbf{f}} = \frac{\mathbf{K}_{\text{curr},\mathbf{f}}^l \cdot \mathbf{K}_{\text{ref}}^l}{\|\mathbf{K}_{\text{curr},\mathbf{f}}^l\| \|\mathbf{K}_{\text{ref}}^l\|} \in \mathbb{R}^T \quad (2)$$

We select the top k tokens with the lowest similarity scores as the dynamic set:

$$\mathcal{I}_{\mathbf{f}} = \arg \text{top-}k (1 - S_{\mathbf{f}}[i])_{i \in \{1 \dots T\}} \quad (3)$$

where $k = \lfloor T \cdot r \rfloor$ and r is determined by R_{Cacher} .

(ii) Selective Attention: We compute the Query and Key only for dynamic tokens indexed by $\mathcal{I}_{\mathbf{f}}$:

$$Q_{\text{sel},\mathbf{f}}^l = Q(\text{LN}_1(\mathbf{X}_{\mathbf{f}}^l))[\mathcal{I}_{\mathbf{f}}], V_{\text{sel},\mathbf{f}}^l = V(\text{LN}_1(\mathbf{X}_{\mathbf{f}}^l))[\mathcal{I}_{\mathbf{f}}] \quad (4)$$

We construct the full Key by initializing it from the cached reference and **scattering** the newly computed Key values:

$$\bar{V}_{\mathbf{f}}^l \leftarrow \mathbf{V}_{\text{ref}}^l; \quad \bar{V}_{\mathbf{f}}^l[\mathcal{I}_{\mathbf{f}}] \leftarrow V_{\text{sel},\mathbf{f}}^l \quad (5)$$

Attention is then computed using the selective queries $Q_{\text{sel},\mathbf{f}}^l$ and the full Key matrix $\bar{V}_{\mathbf{f}}^l$:

$$\mathbf{A}_{\text{sel},\mathbf{f}}^l = \text{Attention}(Q_{\text{sel},\mathbf{f}}^l, \bar{K}_{\mathbf{f}}^l, \mathbf{V}_{\text{curr},\mathbf{f}}^l) \quad (6)$$

(iii) Scatter-Update Output: The attention output is updated by scattering the results into the cached attention $\mathbf{A}_{\text{ref}}^l$:

$$\bar{A}_{\mathbf{f}}^l \leftarrow \mathbf{A}_{\text{ref}}^l; \quad \bar{A}_{\mathbf{f}}^l[\mathcal{I}_{\mathbf{f}}] \leftarrow \mathbf{A}_{\text{sel},\mathbf{f}}^l \quad (7)$$

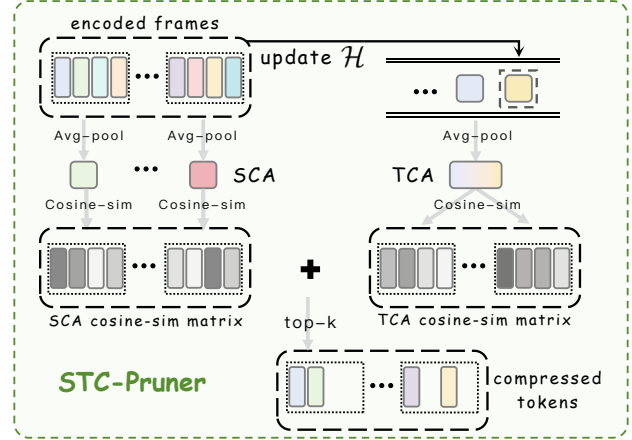


Figure 6. **The Mechanism of STC-Pruner.** To accelerate LLM prefilling, STC-Pruner scores each token based on its novelty. Novelty is measured as the joint dissimilarity to a **Temporal Context Anchor** (TCA), representing historical context, and a **Spatial Context Anchor** (SCA), representing the current frame’s global context. Only tokens with high novelty scores are retained.

This updated tensor is passed to the MLP block, where a similar process is applied.

As depicted in Figure 4, by focusing computation on dynamic tokens and reusing cached values for static content, STC-Cacher significantly reduces the computational load of ViT encoding in streaming scenarios. In this way, STC-Cacher preserves the temporal information content while maintaining efficient processing, leading to reduced encoding time and memory usage.

3.4. STC-Pruner: Dual-Anchor Pruning for LLM

Despite an accelerated ViT by our STC-Cacher, the LLM prefilling stage remains a bottleneck due to the long context sequence Z and the quadratic complexity of self-attention. Existing token pruning methods [27, 42, 63] are often designed for offline, query-aware settings, making them unsuitable for streaming inference where decisions must be made without knowledge of the user query or future frames. This leads to our second research question:

In a query-agnostic and future-agnostic streaming context, what principle can serve as a reliable proxy for token importance to guide token pruning?

To address this, we introduce **STC-Pruner** (Figure 6), a **dual-anchor pruning** strategy to reduce context length for efficient LLM prefilling. STC-Pruner prunes tokens redundant to both the accumulated history and the current frame’s context. The process is below:

(I) Anchor Establishment: We establish two anchors to model the past and present context in streaming videos, guiding the whole token pruning process:

- **Temporal Context Anchor (TCA):** The mean of the historical buffer, representing the aggregated context of the

Methods	Real-Time Visual Perception							Backward Tracing				Forward Active Respond.				Overall	ViT Enc. Latency	LLM Pref. Latency
	OCR	ACR	ATR	STU	FPD	OJR	Avg.	EPM	ASI	HLD	Avg.	REC	SSR	CRR	Avg.			
End-to-End Online VideoLLMs with STC-Cacher																		
Dispider _(CVP25)	49.7	44.0	60.3	43.8	59.4	48.4	51.0	47.8	54.7	5.4	36.0	16.9	40.1	45.8	34.3	40.4	26.4	115.9
+STC-Cacher	50.7	39.4	55.9	41.7	58.4	48.5	49.1	47.5	53.7	5.4	35.2	17.2	36.6	45.3	33.4	39.2	18.9 _(↓28.4%)	115.9
LiveCC _(CVP25)	68.5	39.4	62.1	43.8	68.3	59.8	57.0	58.2	59.5	89.2	69.0	30.9	56.4	72.1	53.2	59.7	181.2	818.4
+STC-Cacher	61.7	36.7	59.5	43.8	67.3	53.8	53.8	56.6	56.1	87.6	66.8	29.8	54.2	70.0	51.3	57.3	126.84 _(↓30.0%)	818.4
StreamForest _(NIPS25)	71.8	51.4	72.4	46.6	67.3	59.8	61.6	59.3	63.5	33.3	52.0	32.5	70.8	56.7	53.3	54.3	103.7	366.2
+STC-Cacher	66.4	49.5	65.5	46.1	66.3	60.9	59.1	57.9	62.2	32.8	51.5	29.9	68.2	56.3	51.5	52.3	67.7 _(↓34.7%)	366.2
Offline-to-Online Framework ReKV with Token Compression Methods																		
ReKV _(ICLR25)	73.8	56.0	74.1	51.7	70.3	60.3	64.4	54.2	57.4	28.5	46.7	25.4	64.6	53.3	47.8	52.6	103.7	482.4
+ToMe _(ICLR23)	61.1	49.5	52.6	42.7	62.4	50.0	53.1	49.2	50.0	29.6	42.9	19.2	60.7	50.0	43.3	46.4	70.5 _(↓32%)	257.8 _(↓46.6%)
+VisionZip _(CVP25)	49.0	49.5	64.7	44.4	64.4	50.5	53.8	47.1	54.1	30.7	44.0	21.9	58.4	53.8	44.7	47.5	103.7	258.3 _(↓46.5%)
+VidCom ² _(EMLP25)	65.8	59.6	69.0	47.2	64.4	56.5	60.4	50.5	53.4	32.8	45.6	25.8	59.0	50.8	45.2	50.4	103.7	259.1 _(↓46.3%)
+STC-Pruner	64.4	59.6	68.1	48.9	65.4	56.5	60.5	51.2	52.0	33.3	45.5	25.9	59.3	52.1	45.8	50.6	103.7	259.2 _(↓46.3%)
+STC-Cacher & Pruner	68.5	57.8	73.3	47.2	68.3	59.8	62.5	50.5	55.4	30.1	45.3	27.9	63.3	52.9	48.0	52.0	78.3 _(↓24.5%)	263.7 _(↓45.3%)

Table 1. **Comprehensive evaluation results on OVO-Bench across three categories:** (i) *Real-Time Visual Perception* (OCR: Optical Character Recognition, ACR: Action Recognition, ATR: Attribute Recognition, STU: Spatial Understanding, FPD: Future Prediction, OJR: Object Recognition), (ii) *Backward Tracing* (EPM: Episodic Memory, ASI: Action Sequence Identification, HLD: Hallucination Detection), (iii) *Forward Active Responding* (REC: Repetition Event Count, SSR: Sequential Steps Recognition, CRR: Clues Reveal Responding). “ViT Enc. Latency” is the time (s) required for ViT to encode 16 frames, and “LLM Pref. Latency” is the time (s) required for LLM prefilling.

recent past in streaming, is $a_{\text{temporal}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} h$, where $\mathcal{H} = \{h_1, \dots, h_{t-1}\}$ is the buffer containing the mean token vectors of the past W frames.

- **Spatial Context Anchor (SCA):** The mean of the token set from the *current* frame, representing its global context or background information, is given by $a_{\text{spatial}} = \frac{1}{N} \sum_{i=1}^N z_i$, where $Z = \{z_i\}_{i=1}^N$ represents the sequence of N visual tokens for the current frame.

(II) Dynamics Scoring: Each token z_j is scored based on its joint dissimilarity to both anchors. Using cosine distance ($d_{\cos}(u, v) = 1 - \text{sim}(u, v)$), the dynamics score $S(z_j)$ is the product of its distances to the temporal and spatial anchors. This multiplicative formulation prioritizes tokens that are distinct from *both* the past and the current frame:

$$S(z_j) = \alpha \cdot d_{\cos}(z_j, a_{\text{temporal}}) + (1 - \alpha) \cdot d_{\cos}(z_j, a_{\text{spatial}}) \quad (8)$$

where $d_{\cos}$ is the cosine distance between the token z_j and the anchors a_{temporal} and a_{spatial} .

(III) Token Pruning: Given a *pruning ratio* R_{Pruner} , we retain the top- k tokens with the highest novelty scores, where $k = \lfloor N \cdot (1 - R_{\text{Pruner}}) \rfloor$ is determined by the preset pruning ratio R_{Pruner} . For example, with $R_{\text{Pruner}} = 25\%$, we prune 75% of the tokens, retaining the top 25% based on their dynamics scores. The pruned token set $Z' = \{z_j | z_j \in \text{TopK}(Z, k, \text{key} = S)\}$ is then passed to the LLM. After processing, the current frame’s spatial context anchor a_{spatial} is appended to the history buffer $\mathcal{H}$ (and the oldest entry is removed) to update the context for the next time step.

The token pruning process is formulated as:

$$Z' \leftarrow \text{TopK}(Z, k, \text{key} = S) \quad (9)$$

where the Top- k selection retains the tokens with the highest

novelty scores, based on the joint dissimilarity to both the temporal and spatial anchors.

In this way, STC-Pruner reduces the context length by pruning redundant tokens while preserving important temporal and spatial information. By using dual anchors, STC-Pruner ensures that only the most relevant tokens are retained, enabling efficient LLM prefilling and reducing the computational burden in streaming video understanding.

4. Experiments

4.1. Experimental Setup

Benchmarks. We evaluate the proposed STC framework across two benchmark categories: (i) *Streaming video understanding*: We use OVO-Bench [35] and Streaming-Bench [23] to assess performance in streaming scenarios. (ii) *Long video understanding*: We select EgoSchema [32], MLVU-dev [64], and VideoMME [16] to evaluate effectiveness in offline long-video understanding.

Implementations. We evaluate two baseline VideoLLMs: (i) *End-to-End Online Models*: We choose Dispider [36], LiveCC [6], and StreamForest [60], which integrate sequence compression during training. We further accelerate their ViT encoding with STC-Cacher for efficient online inference. (ii) *Offline-to-Online Frameworks*: We select ReKV [11], with LLaVA-OV-7B [19] as the backbone, to evaluate the impact of both STC-Cacher and STC-Pruner on ViT encoding and LLM prefilling efficiency, respectively. All models follow the original settings (0.5 fps protocol).

Baselines. We compare our STC with token compression methods like ToMe [3] for ViT encoding, and VisionZip [55] and VidCom² [27] for LLM prefill efficiency. For STC-Cacher only, we set $\mathcal{N} = 4$ and $R_{\text{Cacher}} = 75\%$ to accelerate

Methods	CS	OP	ATP	PR	ACP	SU	EU	CT	TR	CR	Overall	ViT Enc. Latency	LLM Pref. Latency
End-to-End Online VideoLLMs with STC-Cacher													
StreamForest _(NIPS25)	82.7	83.1	84.3	76.9	75.6	69.1	77.5	54.4	78.2	82.8	77.3	103.7	366.2
+STC-Cacher	82.3	81.7	81.6	78.7	75.6	67.9	78.8	62.2	75.1	81.3	76.9	67.7 _(↓34.7%)	366.2
Offline-to-Online Framework ReKV with Token Compression Methods													
ReKV _(ICLR25)	79.2	77.5	75.6	66.0	62.2	60.3	72.3	43.6	69.7	79.5	69.1	103.7	482.4
+ToMe _(ICLR23)	67.5	64.6	66.3	63.0	58.4	53.3	65.2	19.7	57.3	<u>76.6</u>	59.4	70.5 _(↓32.0%)	257.8 _(↓46.6%)
+VisionZip _(CVPR25)	69.5	66.4	69.3	50.7	52.9	57.2	<u>64.1</u>	33.3	54.8	73.4	60.4	103.7	258.3 _(↓46.5%)
+VidCom ² _(EMNLP25)	76.0	<u>68.1</u>	<u>71.6</u>	62.0	<u>58.6</u>	52.0	64.0	42.5	60.4	<u>76.6</u>	63.6	103.7	259.1 _(↓46.3%)
+STC-Pruner	<u>75.4</u>	66.8	71.2	<u>63.9</u>	57.8	51.2	64.0	45.1	<u>63.2</u>	<u>76.6</u>	<u>63.7</u>	103.7	259.2 _(↓46.3%)
+STC-Cacher & Pruner	74.4	71.3	72.9	69.4	58.9	52.4	60.9	<u>44.0</u>	66.9	77.3	65.2	78.3 _(↓24.5%)	263.7 _(↓45.3%)

Table 2. **Comprehensive evaluation results on StreamingBench across real-time understanding tasks:** (i) *Real-Time Visual Understanding* (OP: Object Perception, CR: Causal Reasoning, CS: Clips Summarization, ATP: Attribute Perception, EU: Event Understanding, TR: Text-Rich Understanding, PR: Prospective Reasoning, SU: Spatial Understanding, ACP: Action Perception, CT: Counting).

Methods	EgoSchema	MLVU-dev	VideoMME				Average
			Short	Medium	Long	Overall	
End-to-End Online VideoLLMs with STC-Cacher							
StreamForest _(NIPS25)	60.7	68.9	76.0	58.6	50.6	61.7	63.8
+STC-Cacher	59.7	68.1	73.0	57.4	51.8	60.7	62.8
Offline-to-Online Framework ReKV with Token Compression Methods							
ReKV _(ICLR25)	57.7	68.6	70.4	55.4	47.3	57.7	61.3
+ToMe _(ICLR23)	55.2	63.1	59.6	52.0	43.4	51.7	56.7
+VisionZip _(CVPR25)	55.8	63.2	59.6	51.8	43.4	51.6	56.9
+VidCom ² _(EMNLP25)	60.6	67.1	68.2	55.7	46.6	56.8	61.5
+STC-Pruner	60.8	67.6	68.7	56.3	46.3	57.1	61.8
+STC-Cacher & Pruner	59.0	67.0	67.3	53.9	48.3	56.5	60.8

Table 3. **Comprehensive evaluation results on three offline long video understanding benchmarks.**

ViT encoding. For STC-Pruner and other methods reducing sequence length, we set $R_{\text{Pruner}} = 75\%$ to compress the sequence to 25%. For STC-Cacher and STC-Pruner together (STC-Cacher & Pruner), we set $\mathcal{N} = 2$ and $R_{\text{Cacher}} = 75\%$, and $R_{\text{Pruner}} = 70\%$ to compress the sequence to 30%.

4.2. Main Comparisons

Table 1, 2, 3 present comprehensive comparisons of our STC framework against other compression methods in streaming and long video understanding scenarios, revealing *three key advantages* of our STC framework:

(i) **Outstanding Performance across Scenarios:** Our STC framework outperforms existing methods on all benchmarks. In Table 1 and 2, compared to the previous state-of-the-art method VidCom² [27], STC improves performance by **1.6** and **1.6** on OVO-Bench [34] and StreamingBench [23], respectively, showcasing the strong performance of STC in streaming scenarios. Additionally, as shown in Table 3, STC-Pruner achieves state-of-the-art results across three long video understanding benchmarks, demonstrating its robust performance across diverse scenarios.

(ii) **Consistent Efficiency Gains:** Both STC-Cacher and STC-Pruner are able to significantly improve ViT encoding and LLM prefilling efficiency. Notably, while both STC-

Cacher and ToMe enhance ViT encoding efficiency, our method outperforms ToMe by **5.6**, **5.8**, and an average of **4.1** across OVO-Bench, StreamingBench, and three offline long video understanding benchmarks, demonstrating optimal performance-efficiency trade-offs. Furthermore, STC reduces ViT encoding latency and LLM prefilling latency by **24.5%** and **45.3%**, respectively, for ReKV on OVO-Bench. (iii) **Model Compatibility:** Thanks to the two complementary compressors STC-Cacher and STC-Pruner, it jointly optimizes ViT encoding and LLM prefilling stages and is plug-and-play for end-to-end online models like Dispider [36], LiveCC [6], and StreamForest [60], accelerating ViT encoding. Additionally, it speeds up both ViT encoding and LLM prefilling stages for offline-to-online framework ReKV [11].

4.3. Ablation Studies and Analysis

We conduct ablation studies on STC-Cacher and STC-Pruner across three subsets of OVObench (EPM, STU, and REC) and an offline long-video benchmark EgoSchema.

Dynamic token evaluation in STC-Cacher. Figure 7 aims to explore optimal strategies for identifying dynamic tokens to determine which can be cached and reused in STC-Cacher.

(i) **Effects of Different Features for Dynamics Evaluation:** We evaluate various features for token dynamics to iden-

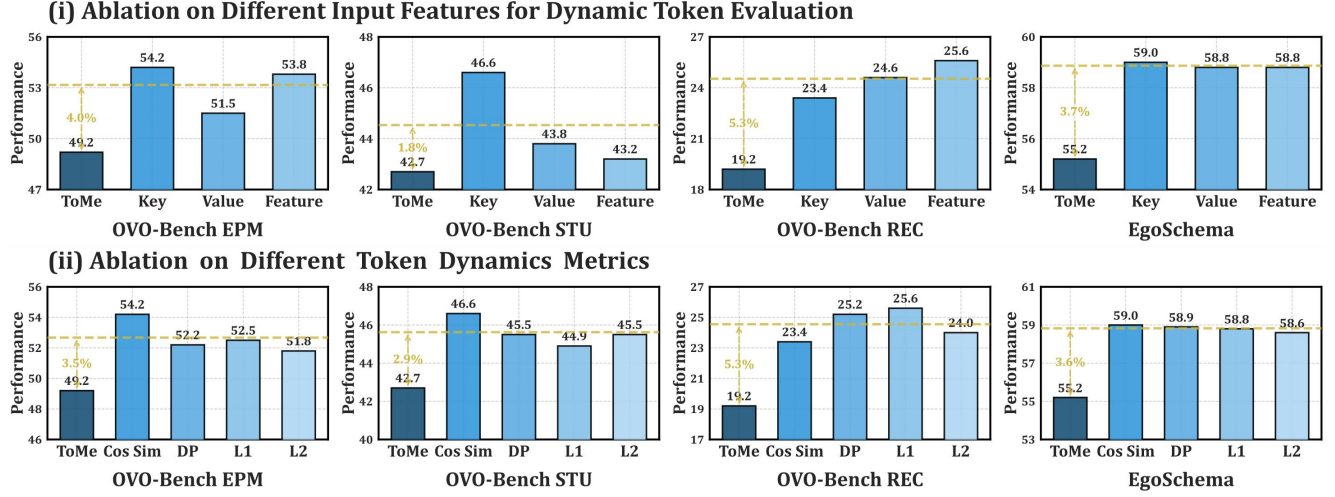


Figure 7. **Effects of Different Token Evaluation Strategies in STC-Cacher.** (i) Compares various features for dynamic token evaluation to identify the optimal dynamic token set for feature caching and reuse. (ii) Further compares different metrics for token dynamics evaluation, with "Cos Sim" referring to cosine similarity (smaller values indicate higher dynamics), "L1" and "L2" representing L1 and L2 distances (smaller values indicate higher dynamics), and "DP" denoting the dot product (smaller values indicate higher dynamics), respectively. The yellow line represents the average performance gap with ToMe [3], indicating that STC-Cacher significantly outperforms ToMe.

Methods	OVO-Bench			EgoSchema
	EPM	STU	REC	
Attn ($R_{\text{Cacher}} = 85\%$)	2.7	2.3	5.3	26.2
MLP ($R_{\text{Cacher}} = 85\%$)	49.8	43.2	25.3	57.1
Attn and MLP ($R_{\text{Cacher}} = 75\%$)	54.2	46.6	23.4	59.0

Table 4. **Effects of different reusing features in STC-Cacher.** "Attn" and "MLP" are reusing features from attention and MLP.

tify the optimal basis. Results show that using key-states yields the best performance, likely because they capture a token’s most informative aspects, including its historical relevance and contribution to attention, making them reliable for dynamic token identification.

(ii) Effects of Different Token Dynamics Metrics: We compare several metrics for token dynamics in ViT. All metrics outperform ToMe, showing that STC-Cacher’s feature caching-reuse strategy accelerates more gently, significantly outperforming ToMe in both streaming and offline settings. Cosine similarity consistently yields the best performance, so it is the default metric in STC-Cacher.

Reusing Features across ViT Blocks. Table 4 investigates which intermediate activations should be reused during selective recomputation. Reusing only attention or MLP activations separately is insufficient: attention-only reuse leads to a performance drop, while MLP-only reuse also underperforms the combined strategy. This suggests that selective recomputation must preserve both the positional/contextual information in attention pathways and the channel/semantic representations in MLP pathways. Our hybrid strategy, reusing both, achieves the right balance, keeping cached tokens informative for downstream reasoning.

Dynamics Scoring for STC-Pruner. Table 5 breaks down the compression criteria of the STC-Pruner scoring mech-

Methods	OVO-Bench			EgoSchema
	EPM	STU	REC	
Only a_{spatial}	50.5	47.2	25.8	59.9
Only a_{temporal}	51.5	47.8	24.1	59.8
a_{spatial} and a_{temporal}	51.2	48.9	25.9	59.9

Table 5. **Effects on different dynamics scoring in STC-Pruner.** a_{spatial} and a_{temporal} are using dynamics scoring by SCA and TCA.

anism. Using either the SCA a_{spatial} or TCA a_{temporal} alone leads to unbalanced performance: the former misses inter-frame novelty, while the latter overlooks intra-frame redundancy. The joint scoring approach, accounting for deviation from both anchors, consistently yields the most robust results. This highlights the need for the pruner to address both intra-frame redundancy and inter-frame dynamics to retain tokens essential for complex reasoning.

5. Conclusion

In this work, we propose Streaming Token Compression (STC), a plug-and-play framework designed to optimize both ViT encoding and LLM pre-filling stages for real-time streaming video understanding. Through two complementary modules, **STC-Cacher** and **STC-Pruner**, STC reduces redundant computations in ViT encoding and compresses token sequences before they enter the LLM, addressing key inefficiencies in streaming video understanding. Our approach significantly enhances processing efficiency while maintaining high accuracy. STC can be easily integrated into existing streaming VideoLLMs without the need for retraining, providing a practical and scalable solution for real-time deployment in latency-sensitive applications.

Acknowledgement

This work was supported by the Shanghai Science and Technology Program (Grant No. 25ZR1402278).

References

- [1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 2, 3
- [2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022. 3
- [3] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *Proceedings of the International Conference on Learning Representations*, 2023. 2, 4, 6, 8
- [4] Joya Chen, Zhaoyang Lv, Shiwei Wu, Kevin Qinghong Lin, Chenan Song, Difei Gao, Jia-Wei Liu, Ziteng Gao, Dongxing Mao, and Mike Zheng Shou. Videollm-online: Online video large language model for streaming video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18407–18418, 2024. 1
- [5] Junjie Chen, Xuyang Liu, Zichen Wen, Yiyu Wang, Siteng Huang, and Honggang Chen. Variation-aware vision token dropping for faster large vision-language models. *arXiv preprint arXiv:2509.01552*, 2025. 1
- [6] Joya Chen, Ziyun Zeng, Yiqi Lin, Wei Li, Zejun Ma, and Mike Zheng Shou. Livecc: Learning video llm with streaming speech transcription at scale. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 29083–29095, 2025. 3, 6, 7, 1
- [7] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *Proceedings of the European Conference on Computer Vision*, 2024. 2, 3
- [8] Yukang Chen, Fuzhao Xue, Dacheng Li, Qinghao Hu, Ligeng Zhu, Xiuyu Li, Yunhao Fang, Haotian Tang, Shang Yang, Zhi-jian Liu, et al. Longvila: Scaling long-context visual language models for long videos. *arXiv preprint arXiv:2408.10188*, 2024. 1
- [9] Yuan Chen, Zichen Wen, Yuzhou Wu, Xuyang Liu, Shuang Chen, Junpeng Ma, Weijia Li, Conghui He, and Linfeng Zhang. Ipcv: Information-preserving compression for mllm visual encoders. *arXiv preprint arXiv:2512.18747*, 2025. 1
- [10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Proceedings of the Advances in Neural Information Processing Systems*, 2022. 2
- [11] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, Hao Jiang, et al. Streaming video question-answering with in-context video kv-cache retrieval. In *Proceedings of the International Conference on Learning Representations*, 2025. 6, 7
- [12] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Tao Zhong, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, and Hao Jiang. Streaming video question-answering with in-context video kv-cache retrieval. *arXiv preprint arXiv:2503.00540*, 2025. 3, 2
- [13] Yue Ding, Yiyang Ji, Jungang Li, Xuyang Liu, Xinlong Chen, Junfei Wu, Bozhou Li, Bohan Zeng, Yang Shi, Yushuo Guan, et al. Omnisift: Modality-asymmetric token compression for efficient omni-modal large language models. *arXiv preprint arXiv:2602.04804*, 2026. 1
- [14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of the International Conference on Learning Representations*, 2021. 1
- [15] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, 2024. 1
- [16] Chaoyou Fu, Yuhang Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24108–24118, 2025. 1, 6
- [17] Yiran Guan, Liang Yin, Dingkan Liang, Jianzhong Ju, Zhenbo Luo, Jian Luan, Yuliang Liu, and Xiang Bai. Video streaming thinking: Videollms can watch and think simultaneously. *arXiv preprint arXiv:2603.12262*, 2026. 1
- [18] Yuhang Han, Xuyang Liu, Zihan Zhang, Pengxiang Ding, Junjie Chen, Honggang Chen, Donglin Wang, Qingsen Yan, and Siteng Huang. Filter, correlate, compress: Training-free token reduction for mllm acceleration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4601–4609, 2026. 3
- [19] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2, 3, 6
- [20] Duo Li, Zuhao Yang, and Shijian Lu. Todre: Visual token pruning via diversity and task awareness for efficient large vision-language models. *arXiv e-prints*, pages arXiv–2505, 2025. 3
- [21] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22195–22206, 2024. 1
- [22] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. Not all patches are what you need: Expediting vision transformers via token reorganizations. In *Proceedings of the International Conference on Learning Representations*, 2022. 4
- [23] Junming Lin, Zheng Fang, Chi Chen, Zihao Wan, Fuwen Luo, Peng Li, Yang Liu, and Maosong Sun. Streamingbench:

- Assessing the gap for mllms to achieve streaming video understanding. *arXiv preprint arXiv:2411.03628*, 2024. 6, 7, 1
- [24] Jihao Liu, Zhiding Yu, Shiyi Lan, Shihao Wang, Rongyao Fang, Jan Kautz, Hongsheng Li, and Jose M Alvarez. Streamchat: Chatting with streaming video. *arXiv preprint arXiv:2412.08646*, 2024. 3
- [25] Xuyang Liu, Xiyan Gui, Yuchao Zhang, and Linfeng Zhang. Mixing importance with diversity: Joint optimization for kv cache compression in large vision-language models. *arXiv preprint arXiv:2510.20707*, 2025. 1
- [26] Xiangrui Liu, Yan Shu, Zheng Liu, Ao Li, Yang Tian, and Bo Zhao. Video-xl-pro: Reconstructive token compression for extremely long video understanding. *arXiv preprint arXiv:2503.18478*, 2025. 3
- [27] Xuyang Liu, Yiyu Wang, Junpeng Ma, and Linfeng Zhang. Video compression commander: Plug-and-play inference acceleration for video large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025. 1, 2, 5, 6, 7
- [28] Xuyang Liu, Zichen Wen, Shaobo Wang, Junjie Chen, Zhisihan Tao, Yubo Wang, Tailai Chen, Xiangqi Jin, Chang Zou, Yiyu Wang, Chenfei Liao, Xu Zheng, Honggang Chen, Weijia Li, Xuming Hu, Conghui He, and Linfeng Zhang. Shifting ai efficiency from model-centric to data-centric compression. *arXiv preprint arXiv:2505.19147*, 2025. 1
- [29] Xuyang Liu, Ziming Wang, Junjie Chen, Yuhang Han, Yingyao Wang, Jiale Yuan, Jun Song, Siteng Huang, and Honggang Chen. Global compression commander: Plug-and-play inference acceleration for high-resolution large vision-language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 7350–7358, 2026. 3
- [30] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. MMBench: Is your multi-modal model an all-around player? In *Proceedings of the European Conference on Computer Vision*, pages 216–233, 2024. 2
- [31] Junpeng Ma, Qizhe Zhang, Ming Lu, Zhibin Wang, Qiang Zhou, Jun Song, and Shanghang Zhang. Mmg-vid: Maximizing marginal gains at segment-level and token-level for efficient video llms. *arXiv preprint arXiv:2508.21044*, 2025. 2
- [32] Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. Egoschema: A diagnostic benchmark for very long-form video language understanding. *Advances in Neural Information Processing Systems*, 36:46212–46244, 2023. 6, 1
- [33] Zhenyu Ning, Guangda Liu, Qihao Jin, Wenchao Ding, Minyi Guo, and Jieru Zhao. LiveVlm: Efficient online video understanding via streaming-oriented kv cache and retrieval. *arXiv preprint arXiv:2505.15269*, 2025. 3
- [34] Junbo Niu, Yifei Li, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, et al. Ovo-bench: How far is your video-llms from real-world online video understanding? In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18902–18913, 2025. 3, 7, 1
- [35] Junbo Niu, Yifei Li, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, et al. Ovo-bench: How far is your video-llms from real-world online video understanding? In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18902–18913, 2025. 6
- [36] Rui Qian, Shuangrui Ding, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Dahua Lin, and Jiaqi Wang. Dispider: Enabling video llms with active real-time interaction via disentangled perception, decision, and reaction. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24045–24055, 2025. 1, 3, 6, 7
- [37] Minghao Qin, Xiangrui Liu, Zhengyang Liang, Yan Shu, Huaying Yuan, Juenjie Zhou, Shitao Xiao, Bo Zhao, and Zheng Liu. Video-xl-2: Towards very long-video understanding through task-aware kv sparsification. *arXiv preprint arXiv:2506.19225*, 2025. 3
- [38] Jiayuan Rao, Haoning Wu, Hao Jiang, Ya Zhang, Yanfeng Wang, and Weidi Xie. Towards universal soccer video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8384–8394, 2025. 1
- [39] Kele Shao, Keda Tao, Can Qin, Haoxuan You, Yang Sui, and Huan Wang. Holitom: Holistic token merging for fast video large language models. In *Proceedings of the Advances in Neural Information Processing Systems*, 2025. 2, 3
- [40] Kele Shao, Keda Tao, Kejia Zhang, Sicheng Feng, Mu Cai, Yuzhang Shang, Haoxuan You, Can Qin, Yang Sui, and Huan Wang. When tokens talk too much: A survey of multimodal long-context token compression across images, videos, and audios. *arXiv preprint arXiv:2507.20198*, 2025. 3
- [41] Yan Shu, Zheng Liu, Peitian Zhang, Minghao Qin, Junjie Zhou, Zhengyang Liang, Tiejun Huang, and Bo Zhao. Video-xl: Extra-long vision language model for hour-scale video understanding. *arXiv preprint arXiv:2409.14485*, 2024. 3
- [42] Keda Tao, Can Qin, Haoxuan You, Yang Sui, and Huan Wang. Dycoke: Dynamic compression of tokens for fast video large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18992–19001, 2025. 3, 5
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 5998–6008, 2017. 4
- [44] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 2, 3
- [45] Shihao Wang, Guo Chen, De-an Huang, Zhiqi Li, Minghan Li, Guilin Li, Jose M Alvarez, Lei Zhang, and Zhiding Yu. Videoitg: Multimodal video understanding with instructed temporal grounding. *arXiv preprint arXiv:2507.13353*, 2025. 1, 2

- [46] Yi Wang, Xinhao Li, Ziang Yan, Yinan He, Jiashuo Yu, Xiangyu Zeng, Chenting Wang, Changlian Ma, Haian Huang, Jianfei Gao, et al. Internvideo2. 5: Empowering video mllms with long and rich context modeling. *arXiv preprint arXiv:2501.12386*, 2025. 1
- [47] Zichen Wen, Yifeng Gao, Shaobo Wang, Junyuan Zhang, Qintong Zhang, Weijia Li, Conghui He, and Linfeng Zhang. Stop looking for important tokens in multimodal language models: Duplication matters more. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025. 1
- [48] Zichen Wen, Shaobo Wang, Yufa Zhou, Junyuan Zhang, Qintong Zhang, Yifeng Gao, Zhaorun Chen, Bin Wang, Weijia Li, Conghui He, et al. Efficient multi-modal large language models via progressive consistency distillation. *arXiv preprint arXiv:2510.00515*, 2025. 1
- [49] Zichen Wen, Yiyu Wang, Chenfei Liao, Boxue Yang, Junxian Li, Weifeng Liu, Haocong He, Bolong Feng, Xuyang Liu, Yuanhuiyi Lyu, et al. Ai for service: Proactive assistance with ai glasses. *arXiv preprint arXiv:2510.14359*, 2025. 1
- [50] Zichen Wen, Boxue Yang, Shuang Chen, Yaojie Zhang, Yuhang Han, Junlong Ke, Cong Wang, Yicheng Fu, Jiawang Zhao, Jiangchao Yao, et al. Innovator-vl: A multimodal large language model for scientific discovery. *arXiv preprint arXiv:2601.19325*, 2026. 1
- [51] Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 28828–28857, 2024. 1
- [52] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2
- [53] Minhao Xiong, Zichen Wen, Zhuangcheng Gu, Xuyang Liu, Rui Zhang, Hengrui Kang, Jiabing Yang, Junyuan Zhang, Weijia Li, Conghui He, et al. Prune2drive: A plug-and-play framework for accelerating vision-language models in autonomous driving. *arXiv preprint arXiv:2508.13305*, 2025. 3
- [54] Haolin Yang, Feilong Tang, Linxiao Zhao, Xiang An, Ming Hu, Huifa Li, Xinlin Zhuang, Boqian Wang, Yifan Lu, Xiaofeng Zhang, et al. Streamagent: Towards anticipatory agents for streaming video understanding. *arXiv preprint arXiv:2508.01875*, 2025. 3
- [55] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 1, 3, 6, 2
- [56] Yanlai Yang, Zhuokai Zhao, Satya Narayan Shukla, Aashu Singh, Shlok Kumar Mishra, Lizhu Zhang, and Mengye Ren. Streammem: Query-agnostic kv cache memory for streaming video understanding. *arXiv preprint arXiv:2508.15717*, 2025. 2, 3
- [57] Zuhao Yang, Sudong Wang, Kaichen Zhang, Keming Wu, Sicong Leng, Yifan Zhang, Bo Li, Chengwei Qin, Shijian Lu, Xingxuan Li, and Lidong Bing. Longvt: Incentivizing “thinking with long videos” via native tool calling. *arXiv preprint arXiv:2511.20785*, 2025. 1
- [58] Zuhao Yang, Yingchen Yu, Yunqing Zhao, Shijian Lu, and Song Bai. Timeexpert: An expert-guided video llm for video temporal grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 24286–24296, 2025. 1
- [59] Linli Yao, Yicheng Li, Yuancheng Wei, Lei Li, Shuhuai Ren, Yuanxin Liu, Kun Ouyang, Lean Wang, Shicheng Li, Sida Li, et al. Timechat-online: 80% visual tokens are naturally redundant in streaming videos. *arXiv preprint arXiv:2504.17343*, 2025. 3
- [60] Xiangyu Zeng, Kefan Qiu, Qingyu Zhang, Xinhao Li, Jing Wang, Jiaxin Li, Ziang Yan, Kun Tian, Meng Tian, Xinhai Zhao, Yi Wang, and Limin Wang. Streamforest: Efficient online video understanding with persistent event memory. In *Proceedings of the Advances in Neural Information Processing Systems*, 2025. 3, 6, 7, 1
- [61] Boqiang Zhang, Kehan Li, Zesen Cheng, Zhiqiang Hu, Yuqian Yuan, Guanzheng Chen, Sicong Leng, Yuming Jiang, Hang Zhang, Xin Li, et al. Videollama 3: Frontier multimodal foundation models for image and video understanding. *arXiv preprint arXiv:2501.13106*, 2025. 2, 3
- [62] Yuanhan Zhang, Jinming Wu, Wei Li, Bo Li, Zejun Ma, Ziwei Liu, and Chunyuan Li. Video instruction tuning with synthetic data. *arXiv preprint arXiv:2410.02713*, 2024. 1, 2, 3
- [63] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. Sparse-VLM: Visual token sparsification for efficient vision-language model inference. In *Proceedings of the International Conference on Machine Learning*, 2025. 2, 3, 5
- [64] Junjie Zhou, Yan Shu, Bo Zhao, Boya Wu, Shitao Xiao, Xi Yang, Yongping Xiong, Bo Zhang, Tiejun Huang, and Zheng Liu. Mlvu: A comprehensive benchmark for multi-task long video understanding. *arXiv preprint arXiv:2406.04264*, 2024. 1, 6